ASSIGNMENT 03

Code:

SinglyLinkedListADT.java { Updated Code}

```
package in.cdac;

public class SinglyLinkedListADT implements List {
	public class Node {
		int data;
		Node next;
	}
	private Node head;
	private Node tail;

	public SinglyLinkedListADT() {
		this.head = null;
		this.tail = null;
	}


	public boolean isEmpty() {
		if(head == null) {
			return true;
		}
		return false;
	}
	@Override
	public void addFirst(int data) {
		Node newNode = new Node();
		newNode.data = data;
		newNode.next = head;
		head = newNode;

		if (tail == null) {
			tail = head;
		}
	}

	@Override
	public void addLast(int data) {
```

```
        Node newNode = new Node();
        newNode.data = data;
        newNode.next = null;

        if(isEmpty()) {
                head = newNode;
                tail = newNode;
        }
        tail.next = newNode;
        tail = newNode;
}

@Override
public int deleteFirst(){
        if(head == null) {
                throw new RuntimeException("The list is Empty. ");
        }
        Node deletedNode = head;
        head = head.next;
        if(isEmpty()) {
                tail = null;
        }
        return deletedNode.data;
}

@Override
public int deleteLast(){
        if(isEmpty()) {
                throw new RuntimeException("The List is Empty");
        }
        Node deletedNode = tail;
        Node tmpNode = head;
        while(tmpNode.next != tail) {
                tmpNode = tmpNode.next;
        }
        tail = tmpNode;
        tmpNode.next = null;

        return deletedNode.data;
}
```

```
// Deletion for the first occurrence
public int deleteByValue(int value) {
	if(isEmpty()) {
		throw new RuntimeException("LinkedList is Empty.");
	}
	Node SearcherNode = head;
	if(head.data == value) {
		int deletedValue = head.data;

		deleteFirst();
		return deletedValue;
	}
	while(SearcherNode.next.data != value) {
		SearcherNode = SearcherNode.next;
		if(SearcherNode.next == null) {
			throw new RuntimeException("Data NOT found.");
		}
	}
	int deletedValue = SearcherNode.next.data;
	SearcherNode.next  = SearcherNode.next.next;
	return deletedValue;
}

// Deleting all the occurrences of value
public void deleteAll(int value) {
	if(isEmpty()) {
		throw new RuntimeException("LinkedList is Empty.");
	}
	Node SearcherNode = head;

	while(SearcherNode != null) {
		if(SearcherNode.data == value) {
			deleteByValue(SearcherNode.data);
		}
		SearcherNode = SearcherNode.next;
	}
}

public boolean searchValue(int value) {
```

```
		Node SearcherNode = head;
		if(head.data == value) {
			return true;
		}
		if(tail.data == value) {
			return true;
		}
		while(SearcherNode.next != null) {
			if(SearcherNode.data == value) {
				return true;
			}
			SearcherNode = SearcherNode.next;

		}
		return false;
	}
	public int getHead() {
		return head.data;
	}
	@Override
	public void displayList(){
		if(isEmpty()) {
			throw new RuntimeException("The List is Empty");
		}
		Node traversalNode = head;
		while(traversalNode != null) {
			System.out.print(traversalNode.data + " ");
			traversalNode = traversalNode.next;
		}
		System.out.println();
	}
}
```

SinglyLinkedListMain.java { Main Program }

```
package in.cdac;

import java.util.Scanner;

public class SinglyLinkedListTest {

	public static void main(String[] args) throws Exception{
		Scanner scanner = new Scanner(System.in);
		int choice;

		SinglyLinkedListADT Slist = new SinglyLinkedListADT();

		do {
			System.out.println("1. Add at First\n"
								+ "2. Add at Last\n"
								+ "3. Delete First\n"
								+ "4. Delete Last\n"
								+ "5. Delete by Value\n"
								+ "6. Delete All Occurrences\n"
								+ "7. Search Element\n"
								+ "8. Display LinkedList\n"
								+ "9. Exit");
			choice = scanner.nextInt();
			switch(choice) {
				case 1: {
						System.out.println("Enter a number to Add at First:
");
						Slist.addFirst(scanner.nextInt());
					}
					break;
				case 2: {
						System.out.println("Enter a number to Add at Last:
");
						Slist.addLast(scanner.nextInt());
					}
					break;
				case 3: {
						System.out.println("Deleted element: " +
Slist.deleteFirst());
```

```
                        }
                        break;
                case 4: {
                                System.out.println("Deleted element: " +
Slist.deleteLast());
                        }
                        break;
                case 5: {
                                System.out.println("Enter a value to be deleted.");
                        System.out.println("Deleted Item:
"+Slist.deleteByValue(scanner.nextInt()));
                }
                break;
                case 6: {
                        System.out.println("Enter a value to be deleted.");
                        Slist.deleteAll(scanner.nextInt());
                }
            break;
                case 7: {
                                System.out.println("Enter a value to be Searched.");
                                        if(Slist.searchValue(scanner.nextInt())) {
                                                System.out.println("Element
Found.");
                                        }
                                        else {
                                                System.out.println("Element NOT
Found");
                                        }
                }
                break;
                case 8: {
                                Slist.displayList();
                        }
                        break;
                case 9: {
                        System.out.println("Exiting.....");
                        }
                        break;
            }
        }while(choice != 9);
```

```
            scanner.close();
        }
}
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
1
Enter a number to Add at First:
15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
1
Enter a number to Add at First:
5
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last:
25
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last:
35
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last:
35
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last:
15
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last:
15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last:
15
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
3
Deleted element: 5
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
4
Deleted element: 15
1. Add at First
2. Add at Last
```

```
Deleted element: 15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
5
Enter a value to be deleted.
15
Deleted Item: 15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
8
```

```
8. Display LinkedList
9. Exit
8
25 35 35 15 15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
6
Enter a value to be deleted.
15
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
8
25 35 35
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
7
Enter a value to be Searched.
35
Element Found.
```

DoublyList.java { Interface }

```
package in.cdac;

public interface DoublyList {
                        public void addFirst(int data);
                        public void addLast(int data);
                        public int deleteFirst();
                        public int deleteLast();
                        public int deleteByValue(int data);
                        public void deleteAll(int data);
                        public void displayList();
}
```

DoublyLinkedListADT.java { Class that implements DoublyList interface }

```
package in.cdac;


public class DoublyLinkedListADT implements DoublyList {
        public class DLLNode{
                int data;
                DLLNode next;
                DLLNode prev;
        }

        private DLLNode head;
        private DLLNode tail;

        public DoublyLinkedListADT() {
                this.head = null;
                this.tail = null;
        }

        private boolean isEmpty() {
           return head == null;
        }

        @Override
        public void addFirst(int data) {

                DLLNode newNode = new DLLNode();
```

```
        newNode.data = data;
        newNode.next = head;
        newNode.prev = null;


        if (isEmpty()) {
      tail = newNode;
   } else {
      head.prev = newNode;
   }
   head = newNode;
}


@Override
public void addLast(int data) {
   DLLNode newNode = new DLLNode();
   newNode.data = data;
   newNode.next = null;

   if (isEmpty()) {
      newNode.prev = null;
      head = newNode;
      tail = newNode;
   } else {
      newNode.prev = tail;
      tail.next = newNode;
      tail = newNode;
   }
}

@Override
public int deleteFirst() {
        if(isEmpty()) {
                throw new RuntimeException("The list is Empty.");
        }
        DLLNode newNode = head;
        head = head.next;
        head.prev = null;
        if(isEmpty()) {
```

```
                tail = null;
        }
        return newNode.data;
}

@Override
public int deleteLast() {
    if(isEmpty())
        throw new RuntimeException("The list is Empty.");

    int data = tail.data;
    if (head == tail) {
        head = tail = null;
    }
    else {
        tail = tail.prev;
        tail.next = null;
    }
    return data;
}
@Override
public int deleteByValue(int value) {
    if (isEmpty())
        throw new RuntimeException("List is Empty.");

    DLLNode current = head;

    while (current != null && current.data != value) {
        current = current.next;
    }

    if (current == null)
        throw new RuntimeException("Value not found.");

    if (current == head)
        return deleteFirst();

    if (current == tail)
        return deleteLast();
```

```
    current.prev.next = current.next;
    current.next.prev = current.prev;

    return current.data;
}

@Override
public void deleteAll(int value) {
    if (isEmpty())
            throw new RuntimeException("The list is empty.");
    DLLNode current = head;
    while (current != null)
    {
        DLLNode nextNode = current.next;
        if(current.data == value)
        {
            if(current == head) {
                    deleteFirst();
            }
            else if(current == tail) {
                    deleteLast();
            }
            else {
                current.prev.next = current.next;
                current.next.prev = current.prev;
            }
        }
        current = nextNode;
    }
}

public int getHead() {
            return head.data;
}


public boolean searchValue(int value) {
    DLLNode current = head;
    while (current != null)
    {
```

```
            if (current.data == value) {
                return true;
            }
            current = current.next;
        }
        return false;
    }


    @Override
    public void displayList() {
        if (isEmpty()) {
            throw new RuntimeException("List is empty.");
        }
        DLLNode current = head;
        System.out.print("Forward:  ");
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println("NULL");
    }

    public void displayReverse() {
        if (isEmpty()) return;

        DLLNode current = tail;
        System.out.print("Backward: ");
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.prev;
        }
        System.out.println("NULL");
    }


}
```

DoublyLinkedListMain.java { Main Program }

```
package in.cdac;
import java.util.Scanner;
public class DoublyLinkedListMain {
	public static void main(String[] args) throws Exception{
		Scanner scanner = new Scanner(System.in);
		int choice;
		DoublyLinkedListADT Slist = new DoublyLinkedListADT();
		do {
			System.out.println("1. Add at First\n"
								+ "2. Add at Last\n"
								+ "3. Delete First\n"
								+ "4. Delete Last\n"
								+ "5. Delete by Value\n"
								+ "6. Delete All Occurrences\n"
								+ "7. Search Element\n"
								+ "8. Display LinkedList\n"
								+ "9. Exit");
			choice = scanner.nextInt();
			switch(choice) {
				case 1: {
						System.out.println("Enter a number to Add at First:
");
						Slist.addFirst(scanner.nextInt());
					}
					break;
				case 2: {
						System.out.println("Enter a number to Add at Last:
");
						Slist.addLast(scanner.nextInt());
					}
					break;
				case 3: {
						System.out.println("Deleted element: " +
Slist.deleteFirst());
					}
					break;
				case 4: {
						System.out.println("Deleted element: " +
Slist.deleteLast());
```

```
                        }
                        break;
                case 5: {
                                System.out.println("Enter a value to be deleted.");
                        System.out.println("Deleted Item:
"+Slist.deleteByValue(scanner.nextInt()));
                }
                break;
                case 6: {
                        System.out.println("Enter a value to be deleted.");
                        Slist.deleteAll(scanner.nextInt());
                }
            break;
                case 7: {
                                System.out.println("Enter a value to be Searched.");
                                        if(Slist.searchValue(scanner.nextInt())) {
                                                System.out.println("Element
Found.");
                                        }
                                        else {
                                                System.out.println("Element NOT
Found");
                                        }
                }
                break;
                case 8: {
                                Slist.displayList();
                        }
                        break;
                case 9: {
                        System.out.println("Exiting.....");
                        }
                        break;
            }
        }while(choice != 9);
        scanner.close();
    }
}
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
8
Forward:  5 15 25 25 35 35 60 NULL
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
1
Enter a number to Add at First:
0
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
2
Enter a number to Add at Last
90
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
3
Deleted element: 0
```

```
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
4
Deleted element: 90
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
5
Enter a value to be deleted.
35
```

```
Deleted Item: 35
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
6
Enter a value to be deleted.
25
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
7
Enter a value to be Searched.
```

```
15
Element Found.
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
8
Forward:  5 15 35 60 NULL
1. Add at First
2. Add at Last
3. Delete First
4. Delete Last
5. Delete by Value
6. Delete All Occurrences
7. Search Element
8. Display LinkedList
9. Exit
```